

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.7

Classes & Objects

Day 7 — building your own types, instead of only using Python's

90 min

Concepts + live code

30 min

Practice notebook

1

New keyword: class

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 6

1 Where does the if go to **FILTER** items?

→ At the end. The result can be shorter.

2 Where does if/else go to **CHOOSE** a value?

→ At the front — and the else is required.

3 What does {n for n in x} build?

→ A set. Add a colon and it becomes a dict.

4 Is (n for n in x) a tuple?

→ No — a generator, and it empties after one pass.

5 When should you write a loop instead?

→ When you cannot read the comprehension aloud.

That closes the data-structures half of Module 1. From today you build your own types.

How today's two hours are structured

1

The idea

Why classes, and the blueprint metaphor

20 min

2

__init__ and self

Building objects that carry their own data

25 min

3

Methods

Functions that belong to an object

25 min

4

Beyond the basics

Class attributes, __str__, when NOT to

15 min

5

Mini build

A BankAccount class

5 min

6

Practice notebook

You code, I walk around

30 min

Almost everything today rests on understanding one word: self.

By the end of today you will be able to

1

Define

Write a class with `__init__` and create objects from it

2

Explain

Say what `self` is, and why every method starts with it

3

Add behaviour

Write methods that read and change an object's own data

4

Decide

Know when a class earns its place over a dict or a function

You have been using objects since Day 1

Every `.append()` on a list, every `.get()` on a dictionary, every `.strip()` on a string was a method call on an object. Today you find out what was behind the dot — and start writing your own. In three weeks `df.head()` will be exactly the same thing.

1

The Idea

Blueprints and instances · 20 minutes

- Why not just a dict?
- class as a blueprint
- Creating objects

Why not just use a dictionary?

Yesterday's approach

```
ravi = {"name": "Ravi", "marks": [88, 71]}
sara = {"name": "Sara", "marks": [91, 84]}

def average(student):
    return sum(student["marks"]) / len(student["marks"])

average(ravi)

# Nothing stops me writing:
ravi["nmae"] = "typo"      # silently accepted
average({"x": 1})          # KeyError at runtime
```

Today's approach

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        return sum(self.marks) / len(self.marks)

ravi = Student("Ravi", [88, 71])
ravi.average()

# The data and what you do with it travel together.
```

One definition

Every Student is built the same way

Data + behaviour

average() lives with the marks it needs

Clear errors

Missing an argument fails immediately

Readable

ravi.average() says what it is

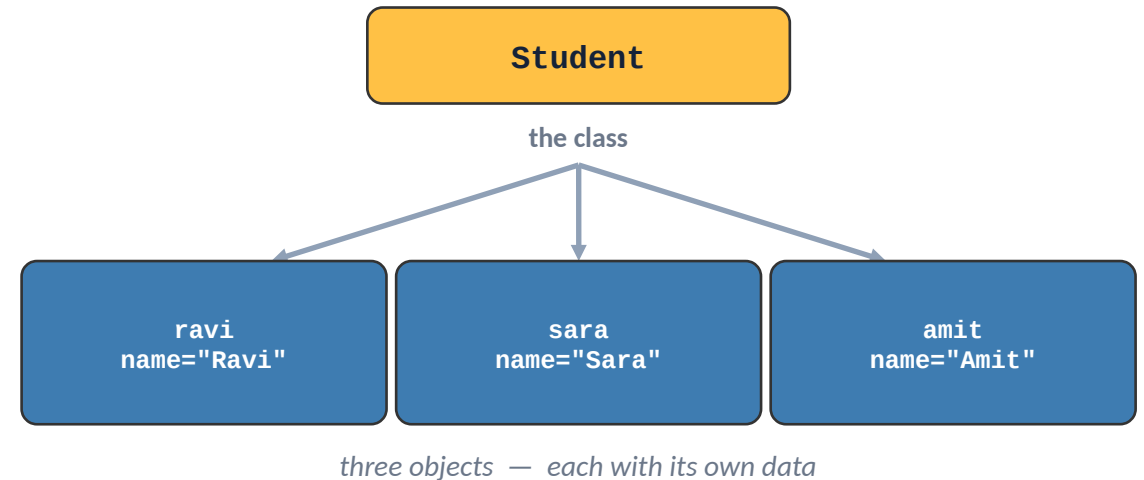
A class is a blueprint; an object is one thing built from it

The blueprint

```
class Student:
    def __init__(self, name):
        self.name = name
```

Defines what every Student will have. Creates nothing on its own — Day 3's “defining is not running”, again.

The objects



Creating an object looks like calling a function — because it is

```
ravi = Student("Ravi")    # calls __init__ for you
```

The class name with brackets after it builds a new object and hands it back. The arguments you pass go straight into `__init__`.



`__init__` and `self`

Where the confusion lives · 25 minutes

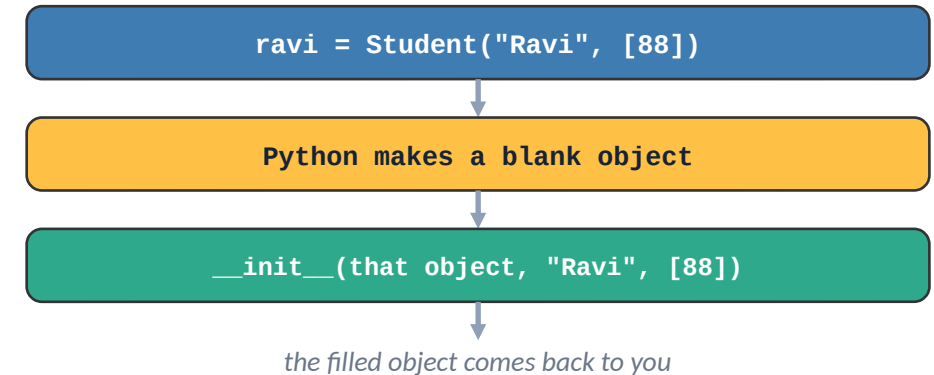
- The initialiser
- What `self` is
- Instance attributes

`__init__` and `self`

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

<code>__init__</code>	runs automatically when you create an object
<code>self</code>	the object being built, handed in by Python
<code>self.name</code>	an attribute stored ON that object
<code>name</code>	just a parameter — gone when <code>__init__</code> ends

What actually happens on creation



self is not magic — it is just the first parameter, filled in for you

```
ravi.average()      # what you write
Student.average(ravi) # what Python actually runs
```

You never pass `self` yourself. Python puts the object before the dot into that first slot automatically. The name `self` is only a convention — Python would accept any name — but every Python programmer expects it, so always use it.

Instance attributes — each object's own data

Every object carries its own copy

```
ravi = Student("Ravi", [88, 71])
sara = Student("Sara", [91, 84])

ravi.name      -> "Ravi"
sara.name      -> "Sara"

ravi.name = "Ravi Kumar"  # change just this one
sara.name      -> "Sara"  unaffected
```

⚠ Three mistakes with self

```
def __init__(name):          # forgot self
    -> TypeError: takes 1 positional
        argument but 2 were given

self.name = name             # correct
name = name                   # does nothing useful

def _init_(self):            # one underscore
    -> never runs. Needs TWO each side.
```

Attributes are just names on an object — you can add them later too

```
ravi.city = "Pune"           # not in __init__ at all
ravi.city                    -> "Pune"
sara.city                    -> AttributeError
```

Python allows this, but it is usually a mistake: now some Students have a city and others do not, and any code that reads `.city` will crash unpredictably. Put everything an object needs in `__init__`, so every object is built the same way.

3

Methods

Functions that belong to an object · 25 minutes

- Defining methods
- Reading state
- Changing state
- Calling other methods

Methods — functions that live inside a class

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def average(self):
        # READS its own data
        return sum(self.marks) / len(self.marks)

    def add_mark(self, mark):
        # CHANGES its own data
        self.marks.append(mark)

    def report(self):
        # calls another method
        return f"{self.name}: {self.average():.1f}"
```

Using them

```
ravi = Student("Ravi", [88, 71])

ravi.average()      -> 79.5

ravi.add_mark(90)
ravi.average()      -> 83.0

ravi.report()
-> "Ravi: 83.0"
```

A method is a normal function with one extra rule

Everything from Day 3 still applies: default arguments, keyword arguments, *args, early return, docstrings. The only difference is that the first parameter is always `self`, and through it the method can reach the object's own data without being handed it. Inside a method, always write `self.marks` — a bare `marks` is a `NameError`.

Reading state versus changing state

Methods that ASK — they return

```
def average(self):  
    return sum(self.marks) / len(self.marks)  
  
def has_passed(self):  
    return self.average() >= 40
```

These leave the object exactly as it was. You use the value they hand back.

Methods that DO — they change self

```
def add_mark(self, mark):  
    self.marks.append(mark)  
  
def rename(self, new_name):  
    self.name = new_name
```

These usually return nothing — Day 4's list methods behave the same way. The change IS the result.

A method that changes state and returns None will bite you

```
ravi = ravi.add_mark(90)      # ravi is now None!  
  
ravi.add_mark(90)            # correct
```

This is exactly Day 4's `nums = nums.append(4)` mistake wearing different clothes. Any method whose job is to change the object should be called on its own line, not assigned. Keep the two kinds separate: a method should either answer a question or change something, not both.

4

Beyond the Basics

Class attributes, printing, judgement · 15 minutes

- Class vs instance
- `__str__`
- When NOT to use a class

Class attributes — shared by every object

Defined in the class body, not in `__init__`

```
class Student:
    school = "Boston University"  # CLASS attribute

    def __init__(self, name):
        self.name = name          # INSTANCE attribute

ravi.school  -> "Boston University"
sara.school  -> "Boston University"  same for all
```

When each one is right

- Instance attribute — anything that differs per object: name, marks, balance
- Class attribute — anything genuinely shared: a constant, a default, a running count of objects created

 **Never make a class attribute a list or dict — the fifth appearance of aliasing**

```
class Student:
    marks = []          # ONE list, shared by ALL

ravi.marks.append(88)
sara.marks  -> [88]     sara got Ravi's mark!
```

One object, many names — Day 1's `c = a`, Day 3's mutable default argument, Day 4's `b = a`, and now this. Mutable data belongs in `__init__`, where each object gets its own: `self.marks = marks` or `self.marks = []`.

Making objects printable, and knowing when to stop

Printing an object, by default

```
print(ravi)
-> <__main__.Student object at 0x7f3a...>
```

Technically correct, completely useless. Add `__str__` and `print()` will use it:

```
def __str__(self):
    return f"Student({self.name})"
```

The dunder methods you meet next

```
__init__    build the object
__str__    what print() shows
__repr__   what the notebook shows
__len__    makes len(obj) work
__eq__     makes obj1 == obj2 work
```

"dunder" = double underscore.
Python calls them for you.

When a class is NOT the answer

Just holding data?

→ a dict is simpler

No data at all, only steps?

→ a function is enough

Only ever one of them?

→ you do not need a blueprint

A class with one method?

→ that method is just a function

A class earns its place when several objects each carry their own data AND there is behaviour that belongs with that data. Both halves matter — data alone is a dictionary, behaviour alone is a function.

Debug corner — eight class bugs

You wrote	What happens	Fix
<code>def __init__(name):</code>	TypeError: 2 args given	<code>def __init__(self, name):</code>
<code>def _init_(self):</code>	never runs, no error	two underscores each side
<code>name = name</code>	attribute never stored	<code>self.name = name</code>
<code>return marks</code>	NameError inside a method	<code>return self.marks</code>
<code>ravi = ravi.add_mark(90)</code>	ravi becomes None	<code>ravi.add_mark(90)</code>
<code>ravi.average</code>	shows the method, does not run it	<code>ravi.average()</code>
<code>marks = []</code> in class body	shared by every object	put it in <code>__init__</code>
<code>print(ravi)</code>	<Student object at 0x...>	define <code>__str__</code>

Rows 1 to 4 are all the same root cause: forgetting that `self` is how a method reaches its own object. All eight are in today's practice notebook.

Putting it together — a BankAccount class

```
class BankAccount:
    """A single account with a balance and a history."""

    bank = "BU Savings"          # class attribute

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
        self.history = []        # own list per object

    def deposit(self, amount):    # CHANGES state
        if amount <= 0:
            return "amount must be positive"
        self.balance += amount
        self.history.append(f"+{amount}")

    def withdraw(self, amount):
        if amount > self.balance:
            return "insufficient funds"
        self.balance -= amount
        self.history.append(f"-{amount}")

    def statement(self):         # ASKS, returns
        return f"{self.owner}: {self.balance} ({len(self.history)} txns)"

    def __str__(self):
        return f"BankAccount({self.owner}, {self.balance})"
```

__init__

owner, balance with a default



Default argument

balance=0 — Day 3



Own list per object

history in __init__, not the body



Class attribute

bank is shared and constant



Changes state

deposit / withdraw return nothing



Asks

statement returns a value



Early return

guards run first — Day 3



__str__

print(acc) reads properly

Every idea from today, plus four from Day 3, in one class.

Practice session — 30 minutes



Predict the output

self, attributes, shared class data

8 questions



Spot & fix the bug

Missing self, __init__, shared list, None

4 questions



Write the code

Rectangle, Book, Counter, Timer, Playlist

5 tasks

Your notebook

Day7_Practice_Questions.ipynb

- Write __init__ first
- Then one method at a time
- Raise a hand — I am walking around
- Solutions shared after class

Two things to check whenever something breaks today: did every method take self, and did you write self.x rather than a bare x?

TODAY IN TWELVE LINES

Recap

- 1 A class is a blueprint; an object is one thing built from it.
- 2 `ClassName(...)` creates an object and runs `__init__`.
- 3 `__init__` needs TWO underscores each side, and `self` first.
- 4 `self` is the object itself, passed in by Python for you.
- 5 `obj.method()` is really `Class.method(obj)`.
- 6 `self.name = name` stores data ON the object.
- 7 Inside a method, always `self.x` — a bare `x` is a `NameError`.
- 8 Methods either ASK (return) or DO (change `self`).
- 9 Never assign the result of a method that changes state.
- 10 Class attributes are shared; never make one a list.
- 11 `__str__` decides what `print(obj)` shows.
- 12 Data only? Use a dict. Steps only? Use a function.

Homework

Write a `Book` class with `title`, `author`, `pages` and a `summary()` method · Add `__str__` to it · Rewrite Day 5's gradebook as a `Gradebook` class

Next class — Topic 1.8

Inheritance & polymorphism: parent and child classes, method overriding, `super()`, and the basics of multiple inheritance